

Implement Client Script & UI Policy (Incident)

Customer journey map · how High-impact Incident controls are discovered, built, tested and extended

TEAM ID

6ab75aba534c8646a9ce4107

Based on project Tasks 1–6: UI Policy, UI Policy Action, onChange / onSubmit / onCellEdit Client Scripts, and testing activities.

SCENARIO Configuring, validating and testing client-side controls on High-impact Incidents																				
Entice How does someone initially become aware of this need?				Enter What do people experience as they begin the configuration?				Engage In the core moments of the build, what happens?				Exit What do people experience as the testing finishes?				Extend What happens after the project is delivered?				
Steps What does the person (or group) typically experience?	Spot data issues Incidents arrive with missing or inconsistent fields	Link to SLAs Bad data hurts triage, routing, reporting and SLA compliance	Define rules High-impact incidents need an owner and a controlled Urgency	Pick the controls UI Policy for behavior, Client Scripts for logic and validation	Log in Open the instance with admin or configuration access	Create UI Policy System UI › UI Policies › New: High Impact Control on Incident, Active	Set condition Impact is 1 – High; Assignment group mandatory; Reverse if false = true	Add Urgency action Urgency Read-only = true; Visible left unchanged	onChange script Impact changes to 1 sets Urgency to 1 and shows an info message	onSubmit script Block save if Impact is High and Assigned To is empty	onCellEdit script Alert and cancel any State edit made from a list	Test mandatory rule New Incident, Impact = High, Assigned To empty, Submit	Test successful save Fill Assigned To and submit; Incident saves with no errors	Test reverse condition Change Impact High to Medium; fields unlock and record saves	Test list-edit block Double-click State in the Incident list; alert appears	Test form update Change State on the form and click Update; change is saved	Cleaner data Incidents are created with complete, valid information	Reliable reporting Consistent fields support reports and SLA tracking	Reuse the pattern Same UI Policy + script approach works on other forms	Wrap up Document results in the project conclusion
	Places: Incident list, reports. People: service desk agents, process owner	Things: SLA and reporting views. People: service delivery manager	People: process owner and admin agreeing the business rules	Things: project brief; skills: UI Policy, Client Scripts, Form Validation	Places: ServiceNow instance. Things: admin / config role	Places: System UI › UI Policies › New	Things: UI Policy form: Conditions section and Reverse if false	Things: UI Policy Actions related list › New	Places: System UI › Client Scripts › New. Things: Type onChange, field Impact	Things: Client Script type onSubmit; g_for m.showErrorBox on Assigned To	Things: Client Script type onCellEdit, field State; alert() and callback(false)	Places: Incident › Create New form	Places: Incident form, Assigned To field, Submit button	Places: Incident › Open; existing High-impact record	Places: Incident › All list view; State cell	Places: same Incident record; Update button	Places: Incident forms and lists	Things: reports, dashboards, SLA tracking	Places: UI Policies and Client Scripts modules	Things: project documentation
Goals & motivations At each step, what is a person's primary goal or motivation? ("Help me..." or "Help me avoid...")	Help me see where manual data entry goes wrong	Help me understand the impact on triage and SLAs	Help me define clear rules for high-impact incidents	Help me choose the right control for each rule	Help me get into the right configuration area	Help me set up a policy that applies only to Incident	Help me trigger controls only when Impact = High	Help me stop users from changing Urgency by mistake	Help me avoid users setting Urgency by hand	Help me avoid incidents saved without an owner	Help me avoid risky State changes in list edits	Help me confirm the rules are really enforced	Help me confirm valid records still save cleanly	Help me avoid fields staying locked after Impact drops	Help me confirm list edits are blocked	Help me confirm normal form updates still work	Help me trust that Incident data is complete	Help me report accurately and meet SLAs	Help me avoid rebuilding this from scratch	Help me keep the solution light and easy to maintain
Positive moments What steps does a typical person find enjoyable, productive, fun, motivating, delightful, or exciting?	Rules are short and easy to state							Urgency updates automatically and an info message explains why	Error box appears right on the Assigned To field			Valid incidents save without errors	Reverse if false unlocks fields automatically	Form updates to State keep working normally		Complete, consistent Incident data	Lightweight, efficient, easy to implement			
Negative moments What steps does a typical person find frustrating, confusing, angering, costly, or time-consuming?	Relying on user awareness leads to incomplete or incorrect data	Reporting accuracy and SLA compliance suffer						Urgency is read-only for High impact; users cannot override it	Save is blocked until Assigned To is filled (intended friction)	List edits of State end in an alert, not a change										
Areas of opportunity How might we make each step better? What ideas do we have? What have others suggested?	How might we back these client-side rules with a server-side Data Policy or Business Rule, since imports and API updates skip them?							How might we align the mandatory field? UI Policy requires Assignment group, onSubmit checks Assigned To	How might we tell users why Urgency is locked?	How might we let authorized roles edit State in lists?				How might we also verify onChange and onSubmit after the reverse?	How might we replace alert() with a friendlier message?			How might we apply the same pattern to Problem and Change?	How might we automate these tests, e.g. with ATF?	